

Implementing WordPiece

This section shows how the **WordPiece algorithm** can be implemented in code.

But one thing is mentioned at the very beginning:

This implementation is made for learning purposes.

That means it is meant to explain the concept.

It is not practical for very large datasets or real-world large corpora.

Which corpus is being used?

Here, the same corpus from the earlier BPE example is used:

```
corpus = [  
    "This is the Building-LLMs-from-scratch-In-Bangla Course.",  
    "This chapter is about tokenization.",  
    "This section shows several tokenizer algorithms.",  
    "Hopefully, you will be able to understand how they are trained and generate tokens.",  
]
```

These are a few sentences from which the WordPiece tokenizer will be trained.

Step 1: First, the text is split into words

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
```

Here, the **bert-base-cased** tokenizer is used.

cased means:

- uppercase and lowercase letters are treated differently
- for example, **H** and **h** are not considered the same

This tokenizer is used for **pre-tokenization**.

That means the full sentence is first broken into smaller words and punctuation marks.

Step 2: Counting how many times each word appears

```
from collections import defaultdict
```

```
word_freqs = defaultdict(int)
for text in corpus:
    words_with_offsets = tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str(text)
    new_words = [word for word, offset in words_with_offsets]
    for word in new_words:
        word_freqs[word] += 1
```

What is happening here?

Each sentence is broken into words, and then the code counts how many times each word appears.

For example, the output may look like this:

```
defaultdict(
  int, {'This': 3, 'is': 2, 'the': 1, ...}
)
```

This means:

- **This** appears 3 times
- **is** appears 2 times
- **the** appears 1 time

So now the tokenizer knows how often each word occurs.

Step 3: Creating the alphabet

At the beginning of WordPiece, the vocabulary is very small.

It contains:

- the first letter of each word
- and the remaining letters of the word with the **##** prefix

This code builds that alphabet:

```
alphabet = []
for word in word_freqs.keys():
```

```
if word[0] not in alphabet:
    alphabet.append(word[0])
for letter in word[1:]:
    if f"#{letter}" not in alphabet:
        alphabet.append(f"#{letter}")
```

For example:

If the word is **Face**

Then it will be split like this:

- **F**
- **##a**
- **##c**
- **##e**

That means:

- the first letter stays plain
- the rest get **##** in front of them

Then the alphabet is sorted:

```
alphabet.sort()
```

Step 4: Adding special tokens

BERT uses some fixed special tokens:

```
vocab = ["[PAD]", "[UNK]", "[CLS]", "[SEP]", "[MASK]"] + alphabet.copy()
```

Their roles in short:

- **[PAD]** → used for padding to make lengths equal
- **[UNK]** → unknown word
- **[CLS]** → special token for classification tasks
- **[SEP]** → sentence separator
- **[MASK]** → used for masked language modeling

These are added at the beginning of the vocabulary.

Step 5: Splitting each word at the character level

```
splits = {  
    word: [c if i == 0 else f"#{c}" for i, c in enumerate(word)]  
    for word in word_freqs.keys()  
}
```

Here, each word is broken into characters.

For example:

`about` → `['a', '##b', '##o', '##u', '##t']`

That means:

- first letter = `a`
- then `##b`, `##o`, `##u`, `##t`

This is the starting point of WordPiece.

Step 6: Computing pair scores

Now the algorithm decides which two tokens should be merged.

For that, this function is used:

```
def compute_pair_scores(splits):  
    letter_freqs = defaultdict(int)  
    pair_freqs = defaultdict(int)  
    for word, freq in word_freqs.items():  
        split = splits[word]  
        if len(split) == 1:  
            letter_freqs[split[0]] += freq  
            continue  
        for i in range(len(split) - 1):  
            pair = (split[i], split[i + 1])  
            letter_freqs[split[i]] += freq  
            pair_freqs[pair] += freq  
            letter_freqs[split[-1]] += freq  
  
    scores = {  
        pair: freq / (letter_freqs[pair[0]] * letter_freqs[pair[1]])  
        for pair, freq in pair_freqs.items()  
    }
```

```
return scores
```

In simple terms, this function does two things:

- counts how often each token appears
- counts how often each pair appears

Then it computes a score.

$$score = \frac{freq_of_pair}{freq_of_first \times freq_of_second}$$

That score helps decide which pair would be the most meaningful one to merge.

Step 7: Finding the best pair based on the score

```
pair_scores = compute_pair_scores(splits)
for i, key in enumerate(pair_scores.keys()):
    print(f'{key}: {pair_scores[key]}')
    if i >= 5:
        break
```

Here, some pair scores are printed.

For example:

```
('T', '##h'): 0.125
('##h', '##i'): 0.03409090909090909
('##i', '##s'): 0.02727272727272727
```

That means the scores were computed for pairs like:

- T + ##h
- ##h + ##i
- ##i + ##s

Then the best pair is searched for:

```
best_pair = ""
max_score = None
for pair, score in pair_scores.items():
```

```
if max_score is None or max_score < score:
    best_pair = pair
    max_score = score
```

Here, the pair with the highest score is selected.

Output:

```
('a', '##b') 0.2
```

That means the first merge will be:

```
a + ##b → ab
```

Step 8: Adding the new merged token to the vocabulary

```
vocab.append("ab")
```

Now `ab` has been added to the vocabulary.

Step 9: Applying the merge inside the splits

It is not enough to only add the token to the vocabulary.
The merge must also be applied inside the existing splits.

For that, this function is used:

```
def merge_pair(a, b, splits):
    for word in word_freqs:
        split = splits[word]
        if len(split) == 1:
            continue
        i = 0
        while i < len(split) - 1:
            if split[i] == a and split[i + 1] == b:
                merge = a + b[2:] if b.startswith("##") else a + b
                split = split[:i] + [merge] + split[i + 2 :]
            else:
                i += 1
        splits[word] = split
    return splits
```

In simple words, it does this:

Wherever **a** and **##b** appear next to each other, it merges them into **ab**.

For example:

```
splits = merge_pair("a", "##b", splits)
splits["about"]
```

Output:

```
['ab', '##o', '##u', '##t']
```

Before, it was:

```
['a', '##b', '##o', '##u', '##t']
```

Now it becomes:

```
['ab', '##o', '##u', '##t']
```

Step 10: Growing the vocabulary by running the loop

```
vocab_size = 70
while len(vocab) < vocab_size:
    scores = compute_pair_scores(splits)
    best_pair, max_score = "", None
    for pair, score in scores.items():
        if max_score is None or max_score < score:
            best_pair = pair
            max_score = score
    splits = merge_pair(*best_pair, splits)
    new_token = (
        best_pair[0] + best_pair[1][2:]
        if best_pair[1].startswith("##")
        else best_pair[0] + best_pair[1]
    )
    vocab.append(new_token)
```

Here, the loop keeps running until the vocabulary size becomes 70.

In each loop:

- scores for all pairs are computed
- the best pair is selected
- that pair is merged
- the new token is added to the vocabulary

This is how WordPiece training continues.

The generated vocabulary

At the end, if you print the vocabulary, you may see something like:

```
['[PAD]', '[UNK]', '[CLS]', '[SEP]', '[MASK]', ..., 'ab', '##fu', 'Fa', 'Fac', ...]
```

The interesting point here is:

WordPiece often learns useful parts of words a bit faster than BPE.

For example:

- Fac
- Hug
- Hugg

These have become meaningful subwords.

An important note

It is mentioned here that if you use `train_new_from_iterator()`, you may not get the exact same vocabulary.

That is because Hugging Face's library does not strictly follow pure WordPiece training internally.

In many cases, it uses a BPE-style approach.

So the code in this tutorial is an approximate implementation meant to explain the logic of WordPiece.

How is a new word tokenized?

This function does that:

```
def encode_word(word):
    tokens = []
    while len(word) > 0:
        i = len(word)
        while i > 0 and word[:i] not in vocab:
            i -= 1
        if i == 0:
            return ["[UNK]"]
        tokens.append(word[:i])
        word = word[i:]
        if len(word) > 0:
            word = f"##{word}"
    return tokens
```

This is very important.

The logic of this function is:

Starting with a word:

- it looks for the largest subword from the beginning that exists in the vocabulary
- it takes that as a token
- then it adds `##` to the remaining part and repeats the same process
- if no valid subword is found, the whole word becomes `[UNK]`

This is the actual tokenization logic of WordPiece.

Example 1: Hugging

```
print(encode_word("Hugging"))
```

Output:

```
['Hugg', '##i', '##n', '##g']
```

That means:

- first, `Hugg` is found
- then `##i`
- then `##n`
- then `##g`

That is how the word gets tokenized.

Example 2: H0gging

```
print(encode_word("H0gging"))
```

Output:

```
[['UNK']]
```

Why?

Because starting from the beginning of the word, no valid sequence of subwords could be found that covers the whole word.

So the whole word becomes unknown.

Function for tokenizing a full text

```
def tokenize(text):
    pre_tokenize_result = tokenizer._tokenizer.pre_tokenizer.pre_tokenize_str(text)
    pre_tokenized_text = [word for word, offset in pre_tokenize_result]
    encoded_words = [encode_word(word) for word in pre_tokenized_text]
    return sum(encoded_words, [])
```

What is happening here?

- first, the sentence is broken into words and punctuation
- then `encode_word()` is applied to each word
- finally, all tokens are joined into one final list

Example: tokenizing a full sentence

```
tokenize("This is the Hugging Face course!")
```

Output:

```
['Th', '##i', '##s', 'is', 'th', '##e', 'Hugg', '##i', '##n', '##g', 'Fac', '##e', 'c', '##o', '##u', '##r', '##s', '##e', '[UNK]']
```

Here:

- This → Th, ##i, ##s
- is → is
- the → th, ##e
- Hugging → Hugg, ##i, ##n, ##g
- Face → Fac, ##e
- course → split into smaller subwords
- ! → [UNK] if it is not in the vocabulary